

LAB_2

Revision of Basic Data Structures

Session Objectives

By the end of this session, you will be able to:

1. Implement and manipulate fundamental data structures
2. Analyze time and space complexity of operations on these structures
3. Select optimal representations for different types of concrete problems
4. Establish effective team collaboration and version control practices

Core Exercises

Exercise 1: Friend Request Timeline

Objective: Manipulate strings and characters to analyze messaging patterns. Social networks process millions of friend requests and messages. Analyzing the content of these requests helps detect spam, harassment, or patterns of toxic behavior. Given a friend request message (string), analyze its sentiment based on character-level patterns.

Requirements:

1. Count the number of uppercase letters in the message (potential indicator of shouting/aggression)
2. Count the number of exclamation marks and question marks (urgency indicators)
3. Calculate the caps ratio = (uppercase letters / total alphabetic characters)
4. Classify the message as:
 - CALM: caps ratio < 0.3 AND punctuation count < 3
 - URGENT: caps ratio ≥ 0.3 OR punctuation count ≥ 3
 - AGGRESSIVE: caps ratio ≥ 0.6 OR excessive punctuation ≥ 5
5. Detect if the message contains repeated characters more than 3 times (e.g., "heyyyyy", "noooo") – potential spam indicator

Examples:

- "Hey, want to connect?" → Uppercase: 1, Punctuation: 1, Caps ratio: 0.11 → CALM
- "PLEASE ACCEPT MY REQUEST!!!" → Uppercase: 3, Punctuation: 3, Caps ratio: 1.0 → AGGRESSIVE
- "Are you free? I need to talk!!!" → Uppercase: 1, Punctuation: 4, Caps ratio: 0.08 → URGENT

Complexity Analysis Questions:

1. How many character comparisons are needed for an n-character string?
2. Can this be done in a single pass? What is the minimal number of passes required?
3. How would the complexity change if we needed to support Unicode characters beyond ASCII?

Exercise 2: Mutual Friends Detection Using Sets

Objective: Implement set operations to find common connections.

Social networks suggest new connections based on mutual friends. Set operations are the natural mathematical foundation for this feature.

Given two users, each with a set of friend IDs, find:

1. Mutual friends (intersection) – people both users know
2. Unique friends of each user (difference)
3. Suggested friends – friends of friends not already connected

Requirements:

1. Represent friend lists as sets
2. Implement functions for:
 - $\text{Intersection}(\text{set1}, \text{set2}) \rightarrow$ mutual friends
 - $\text{Difference}(\text{set1}, \text{set2}) \rightarrow$ friends of user1 not in user2's network
 - $\text{Union}(\text{set1}, \text{set2}) \rightarrow$ all unique friends across both users
3. Calculate the mutual friend coefficient = $|\text{Intersection}| / |\text{Union}|$ (This is the Jaccard similarity of their social circles)
4. For a given user, find friend-of-friend suggestions not already connected (2nd degree connections)

Examples:

- User A friends: {101, 102, 103, 104, 105}
- User B friends: {103, 104, 106, 107, 108}
- Mutual friends: {103, 104}
- Unique to A: {101, 102, 105}
- Unique to B: {106, 107, 108}
- Jaccard similarity: $2/8 = 0.25$

Complexity Analysis Questions:

1. What is the time complexity of set intersection for sets of size m and n?
2. How does the representation (sorted array vs. hash set vs. bit array) affect complexity?
3. For a network with millions of users, which representation would you choose and why?

Exercise 3: Friend Recommendation by Common Interests

Objective: Use multidimensional arrays to represent user-interest matrix.

Social networks recommend friends based on shared interests. This can be modeled as a user-interest matrix where rows = users, columns = interests, cells = interest strength (0–10). Given a matrix of user interests, find potential friend recommendations based on interest similarity.

Requirements:

1. Represent the user-interest matrix as a 2D array of integers (0–10 scale)
 - Rows: users (0..U-1)
 - Columns: interests (0..I-1)
2. Implement cosine similarity to measure interest profile similarity:
$$\text{similarity} = \frac{\text{dot_product}(\text{user_a}, \text{user_b})}{(\text{norm}(\text{a}) * \text{norm}(\text{b}))}$$
3. For a given user, find top K most similar users who are not already friends
4. Implement collaborative filtering: recommend interests a user hasn't rated based on similar users
5. Bonus: Implement dimensionality reduction by grouping correlated interests

Examples:

- Interests: [Music, Sports, Tech, Fashion, Travel, Food]
- User 0: [10, 0, 8, 2, 5, 7] // Loves music, tech, food; hates sports
- User 1: [9, 1, 7, 3, 6, 8] // Very similar to User 0
- User 2: [2, 9, 1, 8, 3, 0] // Very different (sports, fashion lover)
- $\text{Similarity}(\text{User0}, \text{User1}) = 0.94$
- $\text{Similarity}(\text{User0}, \text{User2}) = 0.18$

Complexity Analysis Questions:

1. What is the time complexity of computing all pairwise similarities for U users and I interests?
2. How would you optimize this for sparse matrices (most users have few interests)?
3. What is the space complexity of storing the full matrix vs. sparse representations?

Exercise 4: Mutual Followers Matrix

Objective: Implement social graph as adjacency matrix using 2D boolean array.

For small-to-medium social networks, the adjacency matrix representation enables $O(1)$ edge lookups and efficient set operations.

Implement a directed social graph (followers/following) using a 2D boolean array.

Requirements:

1. Create a FollowerMatrix structure:
 - matrix: 2D boolean array [1..N, 1..N]
 - size: integer (N = maximum users)
 - user_count: actual number of users
2. $\text{matrix}[i][j] = \text{True}$ means user i follows user j
3. Implement functions:
 - Follow(follower, followee): set $\text{matrix}[\text{follower}][\text{followee}] = \text{True}$
 - Unfollow(follower, followee): set to False
 - Is_following(follower, followee): check relationship
 - Get_followers(user): return array of users following this user
 - Get_following(user): return array of users this user follows
4. Implement mutual follow detection: find all pairs where i follows j AND j follows i
5. Implement influence score = (followers count + following count) / total_users

Examples:

- Matrix 3x3:
1 2 3
1 F T F // User1 follows User2
2 T F T // User2 follows User1 and User3
3 F F F // User3 follows no one
- Mutual follows: (1,2) and (2,1)
- Influence: User1=2/3, User2=3/3, User3=0/3

Complexity Analysis Questions:

1. What is the time complexity of Get_followers() and Get_following()?
2. What is the space complexity of the adjacency matrix for N users?
3. At what network size does this become impractical? Why?

Final Question

Give some potential usage of these LAB_2 developed algorithms in your future semester project.

General Instructions

Repository Setup

Create branch: LAB_02_Revision

File Structure:

Follow the same structure of LAB_01

Git Requirements:

Same as for LAB_01

Documentation & Notation

At the End of the Session	Before Friday Midnight of the Same Week
Note: 10	Note: 10
• Each Team (of number BB) Upload at the end of the session a very short PDF document and name it TBB_LAB2_AES.pdf, containing for each exercise: the algorithm in Pseudo-Code (according to the rules explained in lecture 1) – Brief answers of the questions, • You can implement your algorithms to make sure that are working, • NO AI-generated content (will be checked for authenticity), • NO code snippets.	• You finish all the implementation of all exercises, • You finalize the previous report and give it a new name TBB_LAB2_BFM.pdf and upload it before Friday midnight of the same week, containing for each exercise: same as previous + more complexity analysis and reflection + Test set for edge cases of your Algorithms' implementation, • AI-generated content is not recommended, • A link to the Team's Repository in GitHub.